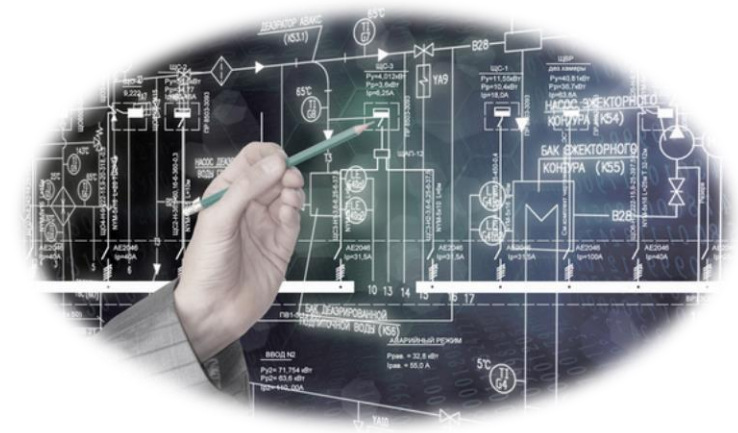


Neural Networks

Regression and Classification Models



* 교재 슬라이드를 수정하여 작성됨

Deep Neural Networks

- How is deep learning different from neural networks?

The “deep” in deep learning is referring to **the depth of layers in a neural network**. A neural network that consists of **more than three layers**—which would be inclusive of the inputs and the output—can be considered a deep learning algorithm.

- How is deep learning different from machine learning?

Deep learning **automates much of the feature extraction piece of the process**, eliminating some of the manual human intervention required. It also enables the use of large data sets referring “scalable learning”. This capability will be particularly interesting as we begin to explore the use of unstructured data more.

Regression with D.N.N

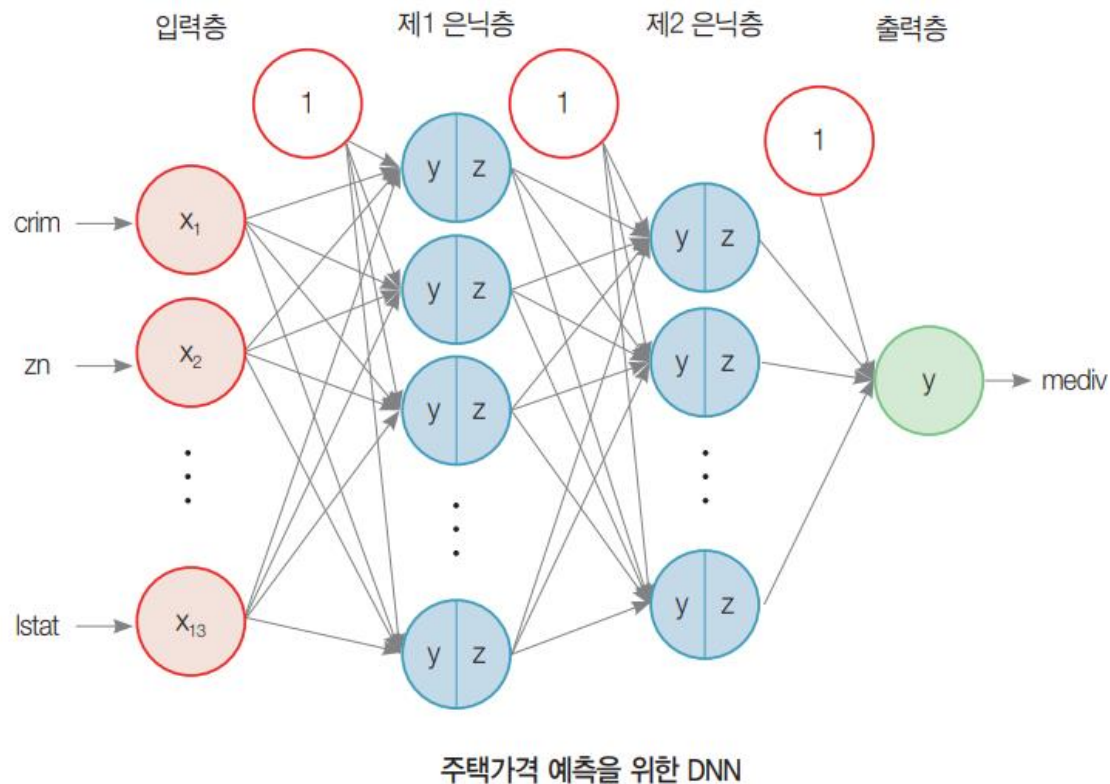
- Dataset: Boston Housing
 - ✓ The problem of predicting house prices with multiple inputs that affect house prices.
 - ✓ 13 Variables with 506 samples
 - House prices (mediv) in Boston neighborhoods in the 1970s and the house prices (mediv) and factors affecting house prices

Var	Definition	Var	Definition
MEDV	median value in \$1000	AGE	% built before 1940
DIS	distance to employment centers	CRIM	per capita crime rate
RAD	accessibility to radial highways	ZN	% land zoned for lots
INDUS	% nonretail business	TAX	property tax/\$10,000
CHAS	1 on Charles River, 0 else	P/T	pupil/teacher ratio
NOX	nitrogen oxide conc. (p.p.10 ⁹)	B	(% black - 63) ² /10
RM	average number of rooms	LSTAT	% lower-status pop.



Regression with D.N.N

- For nonlinear regression, DNN structure requires more than one hidden layer
→ Three hidden layers will be used
- House price(medv) as output, the rest variables as input
 - ✓ Exclude 'chas' variable (discrete numerical data)



Regression with D.N.N

- ReLU as activation function
- **In case of regression, output node should not have any activation function!**

- MSE as a loss function

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

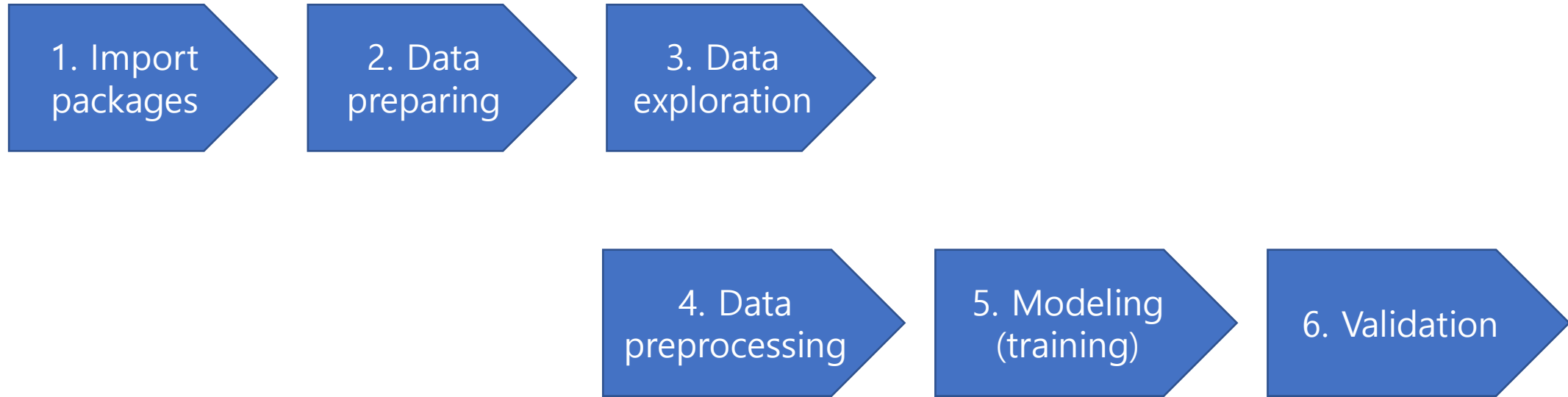
- n samples
- i: index of samples
- y_i : ground truth
- $\hat{y}_i$: predicted value

vs. MAE as a model performance metric

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- ADAM as an optimizer

Procedure of NN modeling



Regression with D.N.N

1. Import packages

```
from sklearn.datasets import load_boston
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error
from keras.models import Sequential
from keras.layers import Dense
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
```

Regression with D.N.N

2. Data preparing

```
# 데이터 불러오기
boston = load_boston()
print(boston)
```

```
{'data': array([[6.3200e-03, 1.8000e+01, 2.3100e+00, ..., 1.5300e+01, 3.9690e+02,
 4.9800e+00],
 [2.7310e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9690e+02,
 9.1400e+00],
 [2.7290e-02, 0.0000e+00, 7.0700e+00, ..., 1.7800e+01, 3.9283e+02,
 4.0300e+00],
 ...,
 [6.0760e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
 5.6400e+00],
 [1.0959e-01, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9345e+02,
 6.4800e+00],
 [4.7410e-02, 0.0000e+00, 1.1930e+01, ..., 2.1000e+01, 3.9690e+02,
 7.8800e+00]]), 'target': array([24. , 21.6, 34.7, 33.4, 36.2, 28.7, 22.9,
 18.9, 21.7, 20.4, 18.2, 19.9, 23.1, 17.5, 20.2, 18.2, 13.6, 19.6,
 15.2, 14.5, 15.6, 13.9, 16.6, 14.8, 18.4, 21. , 12.7, 14.5, 13.2,
 13.1, 13.5, 18.9, 20. , 21. , 24.7, 30.8, 34.9, 26.6, 25.3, 24.7,
 21.2, 19.3, 20. , 16.6, 14.4, 19.4, 19.7, 20.5, 25. , 23.4, 18.9,
 35.4, 24.7, 31.6, 23.3, 19.6, 18.7, 16. , 22.2, 25. , 33. , 23.5])}
```

data 부분

target 부분

Regression with D.N.N

2. Data preparing

```
# 입력 데이터의 데이터 프레임
```

```
X = pd.DataFrame(boston.data, columns=boston.feature_names)  
print(X)
```

	CRIM	ZN	INDUS	CHAS	NOX		RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.0	0.538	...	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.0	0.469	...	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.0	0.469	...	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.0	0.458	...	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.0	0.458	...	3.0	222.0	18.7	396.90	5.33
...	...	...	...	...	...	...	...	...	...	...	...
501	0.06263	0.0	11.93	0.0	0.573	...	1.0	273.0	21.0	391.99	9.67
502	0.04527	0.0	11.93	0.0	0.573	...	1.0	273.0	21.0	396.90	9.08
503	0.06076	0.0	11.93	0.0	0.573	...	1.0	273.0	21.0	396.90	5.64
504	0.10959	0.0	11.93	0.0	0.573	...	1.0	273.0	21.0	393.45	6.48
505	0.04741	0.0	11.93	0.0	0.573	...	1.0	273.0	21.0	396.90	7.88

```
[506 rows x 13 columns]
```

Regression with D.N.N

2. Data preparing

```
# CHAS 열 삭제  
X = X.drop(['CHAS'],axis=1)  
print(X.head())
```

	CRIM	ZN	INDUS	NOX	RM	...	RAD	TAX	PTRATIO	B	LSTAT
0	0.00632	18.0	2.31	0.538	6.575	...	1.0	296.0	15.3	396.90	4.98
1	0.02731	0.0	7.07	0.469	6.421	...	2.0	242.0	17.8	396.90	9.14
2	0.02729	0.0	7.07	0.469	7.185	...	2.0	242.0	17.8	392.83	4.03
3	0.03237	0.0	2.18	0.458	6.998	...	3.0	222.0	18.7	394.63	2.94
4	0.06905	0.0	2.18	0.458	7.147	...	3.0	222.0	18.7	396.90	5.33

[5 rows x 12 columns]

Regression with D.N.N

2. Data preparing

```
# 입력 변수의 정보  
print(X.info())
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 506 entries, 0 to 505  
Data columns (total 12 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
0    CRIM        506 non-null    float64  
1    ZN          506 non-null    float64  
2    INDUS       506 non-null    float64  
3    NOX         506 non-null    float64  
4    RM          506 non-null    float64  
5    AGE         506 non-null    float64  
6    DIS         506 non-null    float64  
7    RAD         506 non-null    float64  
8    TAX         506 non-null    float64  
9    PTRATIO     506 non-null    float64  
10   B           506 non-null    float64  
11   LSTAT       506 non-null    float64  
dtypes: float64(12)  
memory usage: 47.6 KB  
None
```

Regression with D.N.N

2. Data preparing

```
# 목표변수의 데이터 프레임  
y = pd.DataFrame(boston.target)  
print(y)
```

```
0      24.0  
1      21.6  
2      34.7  
3      33.4  
4      36.2  
...      ...  
501     22.4  
502     20.6  
503     23.9  
504     22.0  
505     11.9
```

```
[506 rows x 1 columns]
```

Regression with D.N.N

3. EDA

```
# 입출력 데이터 세트의 데이터 프레임
boston_df = pd.DataFrame(data=boston.data, columns=boston.feature_names)
boston_df["MEDIV"] = boston.target
print(boston_df.head())
```

	CRIM	ZN	INDUS	CHAS	NOX	...	TAX	PTRATIO	B	LSTAT	MEDIV
0	0.00632	18.0	2.31	0.0	0.538	...	296.0	15.3	396.90	4.98	24.0
1	0.02731	0.0	7.07	0.0	0.469	...	242.0	17.8	396.90	9.14	21.6
2	0.02729	0.0	7.07	0.0	0.469	...	242.0	17.8	392.83	4.03	34.7
3	0.03237	0.0	2.18	0.0	0.458	...	222.0	18.7	394.63	2.94	33.4
4	0.06905	0.0	2.18	0.0	0.458	...	222.0	18.7	396.90	5.33	36.2

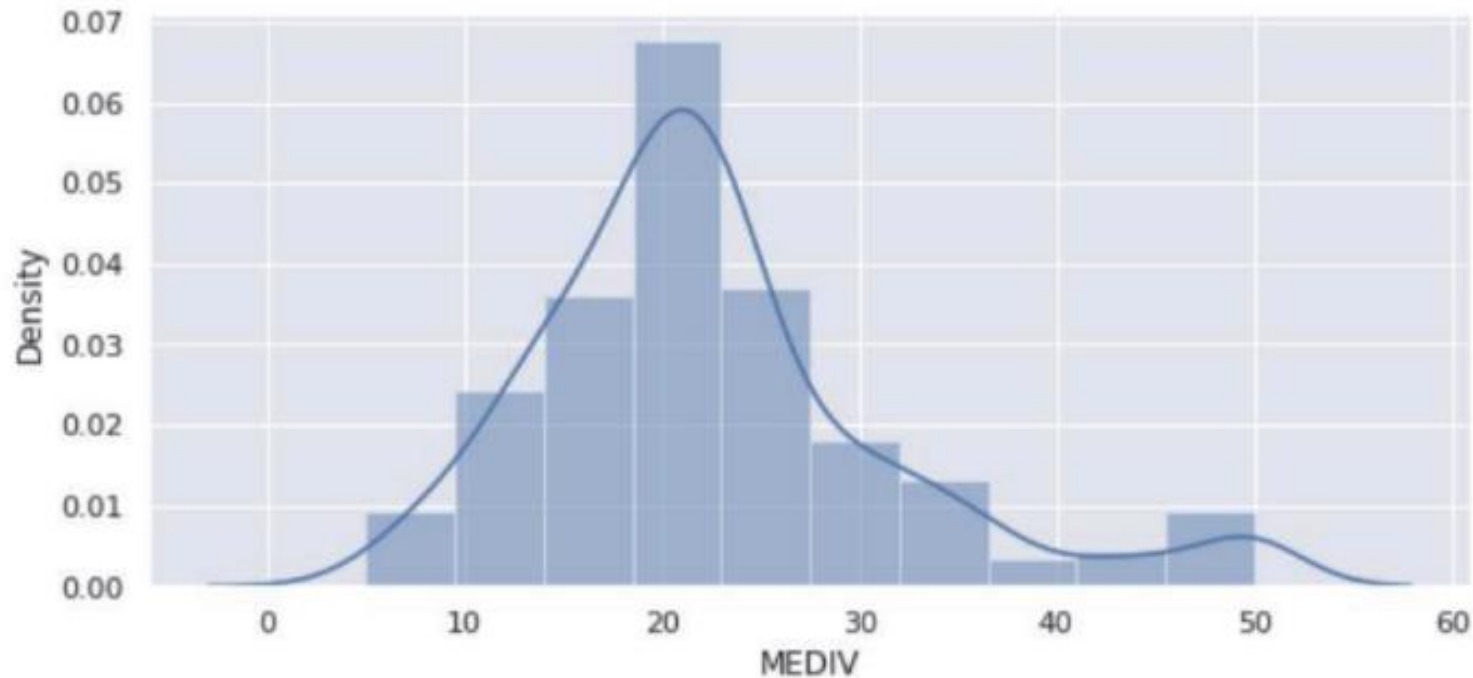
[5 rows x 14 columns]

```
# CHAS 열 삭제
boston_df = boston_df.drop(['CHAS'],axis=1)
boston_df.info()
```

Regression with D.N.N

3. EDA

```
# 목표변수 값의 히스토그램  
sns.set(rc={'figure.figsize':(10, 5)})  
sns.distplot(boston_df['MEDIV'], bins=10)  
plt.show()
```



Regression with D.N.N

3. EDA

```
# 각 변수별 상관계수
correlation_matrix = boston_df.corr().round(2)
# 상관계수의 히트맵
sns.set(rc={'figure.figsize':(10, 8)})
sns.heatmap(data=correlation_matrix, annot=True)
plt.show()
```



Regression with D.N.N

4. Data preproccesing (missing value is omitted)

```
# 학습용과 데이터 분리(7:3)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)
```

Split train and test data

```
# 정규화 스케일러 생성
scalerX = MinMaxScaler()
# 정규화 스케일러를 학습용 데이터에 맞춤
scalerX.fit(X_train)
# 정규화 스케일러로 학습용 데이터를 변환
X_train_norm = scalerX.transform(X_train)
# 정규화 스케일러로 테스트용 데이터를 변환
X_test_norm = scalerX.transform(X_test)
```

MinMax scaling for X and Y

```
# 정규화 스케일러 생성
scalerY = MinMaxScaler()
# 정규화 스케일러를 학습용 데이터에 맞춤
scalerY.fit(y_train)
# 정규화 스케일러로 학습용 데이터를 변환
y_train_norm = scalerY.transform(y_train)
# 정규화 스케일러로 테스트용 데이터를 변환
y_test_norm = scalerY.transform(y_test)
```


Regression with D.N.N

5. Modeling (training)

모형 구조

```
model = Sequential()  
model.add(Dense(60, activation='relu', input_shape=(12,)))  
model.add(Dense(60, activation='relu'))  
model.add(Dense(30, activation='relu'))  
model.add(Dense(1))
```

순차 모형

제1은닉층과 입력층

제2은닉층

제3은닉층

출력층

모형 구조 출력

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 60)	780
dense_5 (Dense)	(None, 60)	3660
dense_6 (Dense)	(None, 30)	1830
dense_7 (Dense)	(None, 1)	31

=====
Total params: 6,301
Trainable params: 6,301
Non-trainable params: 0
=====

제1은닉층의
노드 수

가중치 : $12 \times 60 = 720$ (개)
편향 : 60(개)

Layer

Sequential()

Dense(60, activation='relu', input_shape=(12,))

Dense(60, activation='relu')

Dense(30, activation='relu')

Dense(1)

Description

Defines a simple stack of layers (input → output)

First hidden layer with 60 neurons + defines the input shape (12 features)

Second hidden layer with 60 neurons

Third hidden layer with 30 neurons

Output layer with 1 neuron (e.g., for predicting a single value)

[Keras in Python]

- Keras is a high-level deep learning library written in Python that allows users to build and train neural networks easily and intuitively.
- Originally developed as an independent library, it is now fully integrated into TensorFlow as `tf.keras`, making it one of the most accessible tools for deep learning.
- Keras is designed for rapid experimentation and prototyping, enabling developers and researchers to create powerful machine learning models with minimal code.
- At its core, Keras provides a simple and modular way to construct neural networks.
- The most common way to build a model is using the Sequential API, where layers are added one after another in a linear stack.
- For more complex architectures, Keras also supports a functional API that allows for building models with multiple inputs, outputs, or shared layers.
- Common layers like Dense, Conv2D, and LSTM are available for various types of tasks such as regression, image classification, or sequence modeling.
- To train a model, Keras provides a clean workflow: first, you define the model structure; then you compile it by specifying the optimizer, loss function, and evaluation metrics; finally, you train it using the `.fit()` method.
- The entire training process can be monitored and evaluated easily with built-in tools.

Regression with D.N.N

5. Modeling (training)

```
# 모형 구성 #Define the model how to learn/train
model.compile(optimizer='adam',
              loss='mse',
              metrics=['mae'])
```

```
# 학습
results = model.fit(X_train_norm, y_train_norm,
                   validation_data=(X_test_norm, y_test_norm),
                   epochs=200, batch_size=32)
```

학습 데이터

테스트 데이터

```
Epoch 1/200
18/18 [=====] - 1s 10ms/step - loss: 0.1120 - mae: 0.2579 - val_loss: 0.0432 - val_mae: 0.1577
Epoch 2/200
18/18 [=====] - 0s 4ms/step - loss: 0.0419 - mae: 0.1540 - val_loss: 0.0291 - val_mae: 0.1132
Epoch 3/200
18/18 [=====] - 0s 4ms/step - loss: 0.0285 - mae: 0.1099 - val_loss: 0.0216 - val_mae: 0.1107
```

Regression with D.N.N

5. Modeling (training)

학습 수에 따른 loss 변화

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(results.history['loss'])
```

학습 데이터의 손실

```
plt.plot(results.history['val_loss'])
```

테스트 데이터의 손실

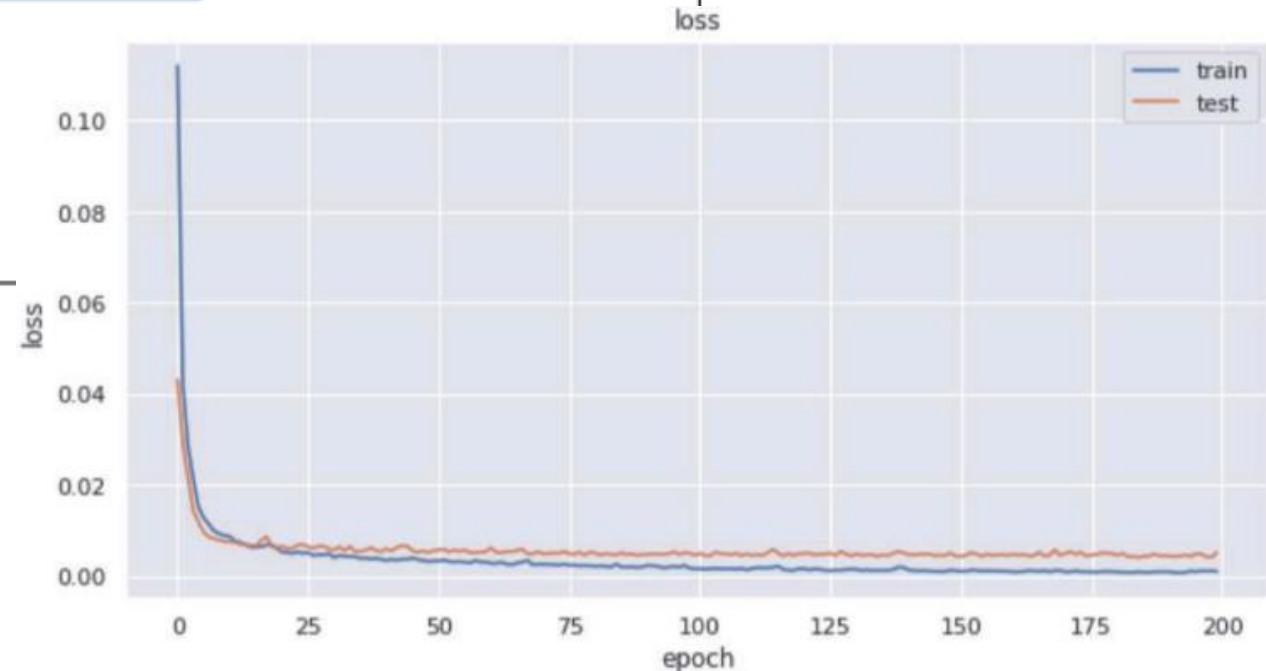
```
plt.title('loss')
```

```
plt.xlabel('epoch')
```

```
plt.ylabel('loss')
```

```
plt.legend(['train', 'test'], loc='upper right')
```

```
plt.show()
```



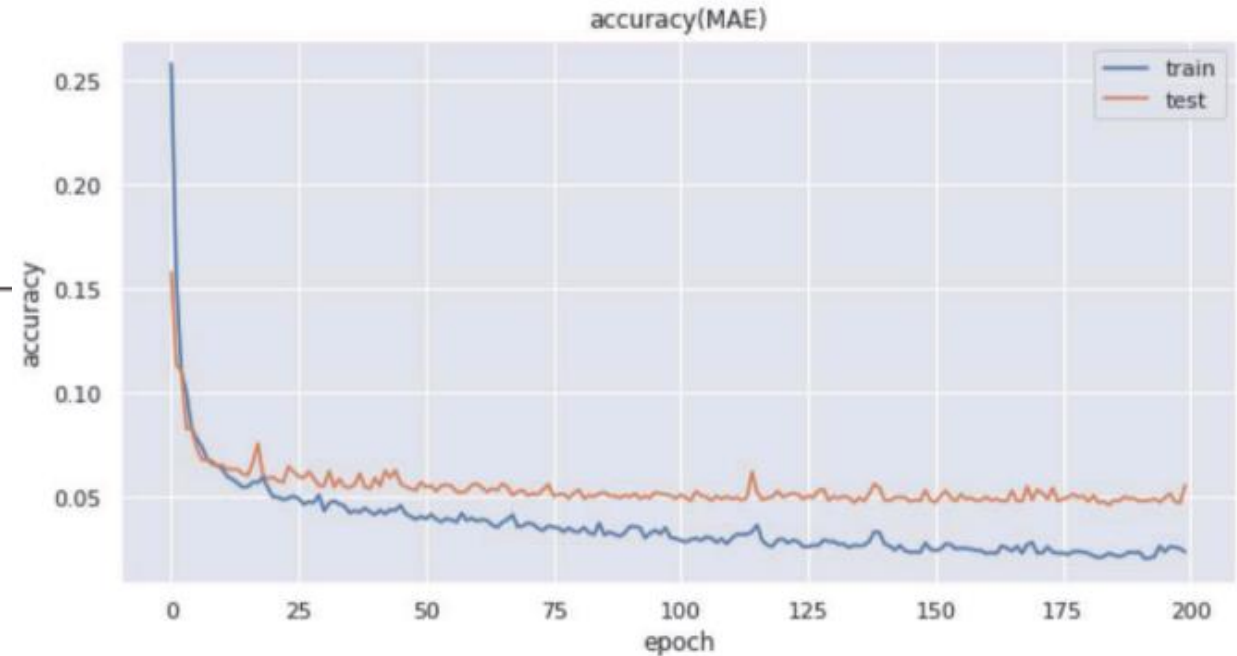
Regression with D.N.N

5. Modelina (training)

```
# 학습 수에 따른 정확도(mae) 변화
plt.figure(figsize=(10, 5))
plt.plot(results.history['mae'])
plt.plot(results.history['val_mae'])
plt.title('accuracy(MAE)')
plt.xlabel('epoch')
plt.ylabel('accuracy')
plt.legend(['train', 'test'], loc='upper right')
plt.show()
```

학습 데이터의 오차

테스트 데이터의 오차



Regression with D.N.N

6. Validation (test)

```
# 테스트 데이터에 대한 예측
y_pred = model.predict(X_test_norm).flatten()
# 예측 값의 역변환
y_pred_inverse = scalerY.inverse_transform(y_pred.reshape(-1,1))
print(y_pred_inverse[0:10])
```

```
[[24.4404 ]
 [23.283087]
 [47.73616 ]
 [22.901867]
 [24.387146]
 [21.155628]
 [18.79687 ]
 [32.0432 ]
 [20.667751]
 [12.652034]]
```

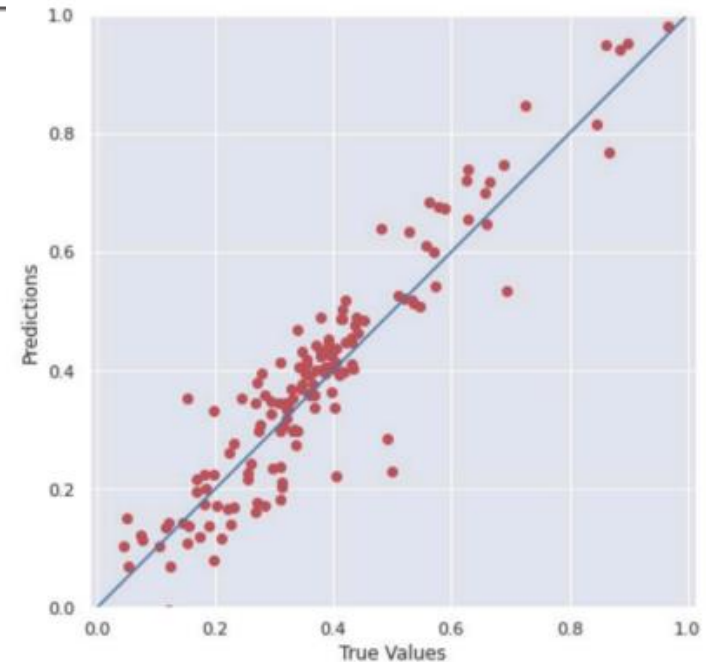
Regression with D.N.N

6. Validation (test)

```
# 오차 측정(MAE)
print('MAE: %.2f' % mean_absolute_error(y_test, y_pred_inverse))
```

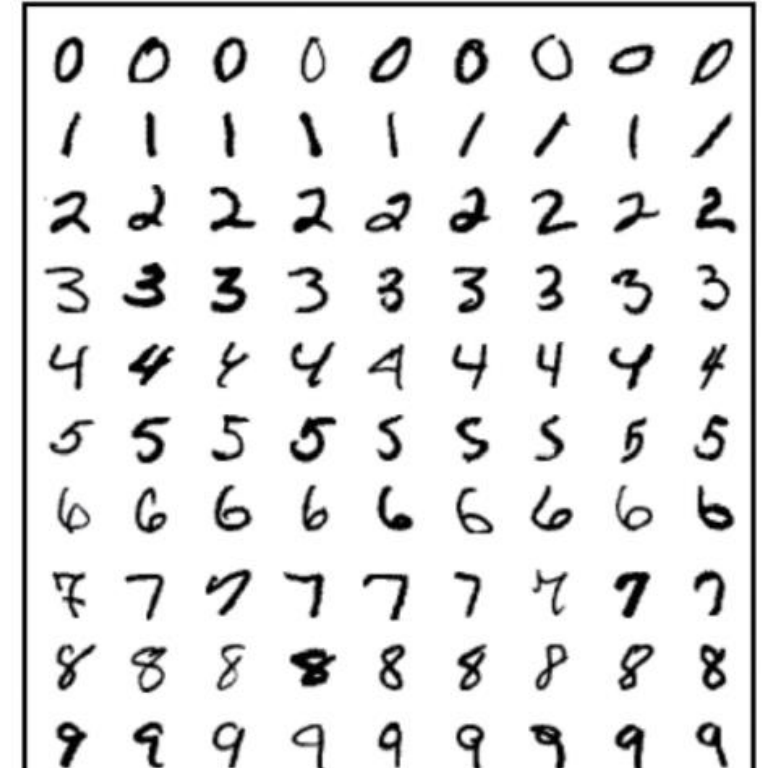
MAE: 2.51

```
# 실제 값 대비 예측 값의 산포도
plt.figure(figsize=(7, 7))
plt.scatter(y_test_norm, y_pred, c='r')
plt.xlabel('True Values')
plt.ylabel('Predictions')
plt.axis('equal')
plt.xlim(0, 1)
plt.ylim(0, 1)
plt.plot([0, 1], [0, 1])
plt.show()
```



Classification with D.N.N

- MNIST is an acronym for Modified National Institute of Standards and Technology
- A set of handwritten digit datasets collected by the U.S. Census Bureau that have been modified to make them suitable for machine learning.
- A grayscale image whose size is 28x28 pixels
- Well-centered and size-uniformed through preprocessing, making them ideal for image processing and machine learning
- 60,000 images of handwriting from 0 to 9
 - ✓ 50,000 for train, 10,000 for test



Classification with D.N.N – Data preparing



tensorflow와 tf.keras를 임포트

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import matplotlib.pyplot as plt
```

MNIST 데이터를 학습용, 테스트 데이터로 구분하여 읽어옴

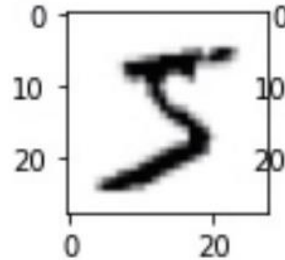
```
mnist = keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = \
mnist.load_data()
```



```
print('train_images.shape =', train_images.shape)
print('test_images.shape =', test_images.shape)
print('train_labels.shape =', train_labels.shape)
print('test_labels.shape =', test_labels.shape)
```

```
train_images.shape = (60000, 28, 28)
test_images.shape = (10000, 28, 28)
train_labels.shape = (60000,)
test_labels.shape = (10000,)
```

Classification with D.N.N – Data preparing

[illegible]

```
train_labels[:4] = [5 0 4 1]
```

Classification with D.N.N – Data preparing

- Normalization, which adjusts the value of the original pixel from the range 0-255 to a value between 0 and 1.



입력값 전처리 : 0에서 1 사이의 값으로 입력 데이터를 가공함

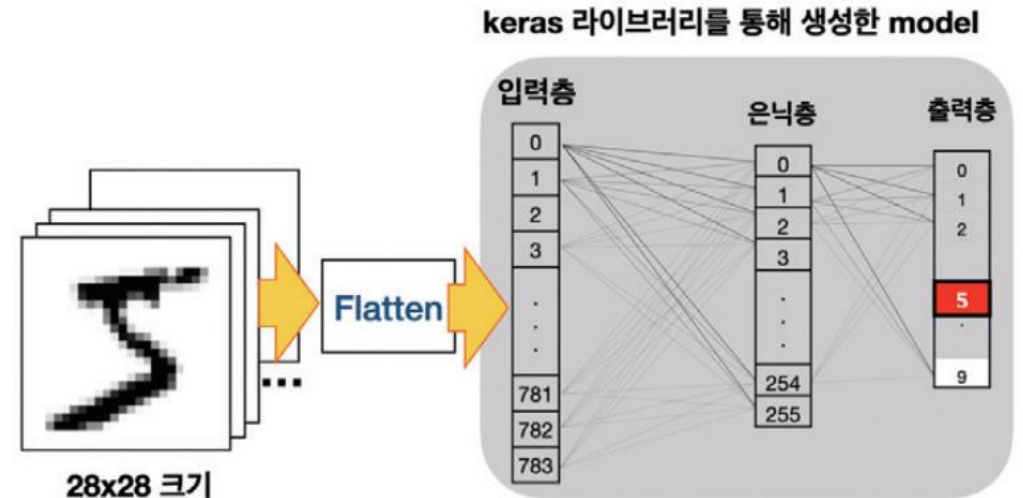
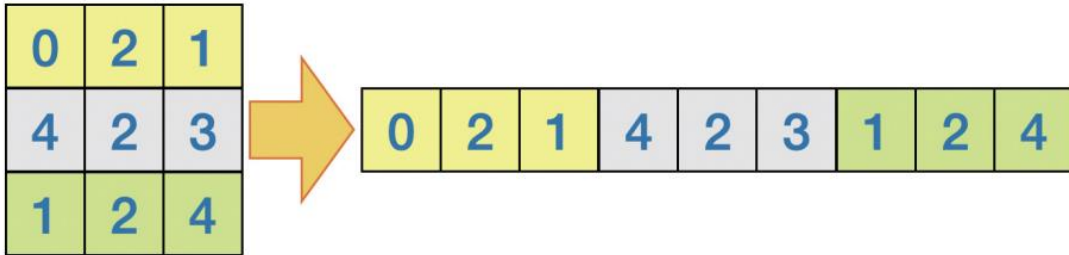
```
train_images, test_images = train_images / 255, test_images / 255
```

Classification with D.N.N – Modeling



```
model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28, 28)),  
    keras.layers.Dense(256, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```

Make two-dimension object to an one-dimensional object using the Flatten()



Classification with D.N.N – Modeling



```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])
```



model.summary() # 모델의 구조를 요약하여 살펴보자

Model: "sequential"

Layer (type)	Output Shape	Param #
flatten (Flatten)	(None, 784)	0
dense (Dense)	(None, 256)	200960
dense_1 (Dense)	(None, 10)	2570

Total params: 203,530

Trainable params: 203,530

Non-trainable params: 0



Classification with D.N.N – Modeling (softmax)



```
model = keras.Sequential([  
    keras.layers.Flatten(input_shape=(28, 28)),  
    keras.layers.Dense(256, activation='relu'),  
    keras.layers.Dense(10, activation='softmax')  
])
```

Number of Class

- Softmax formula:

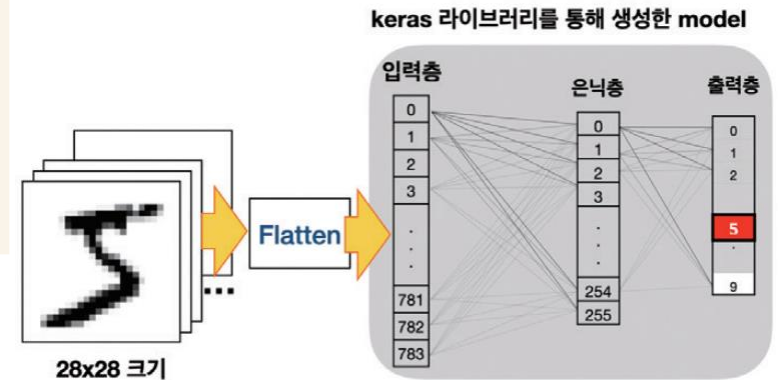
$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

- Example:

$$\begin{bmatrix} 8 \\ 5 \\ 0 \end{bmatrix}$$

softmax()

$$\begin{aligned}\sigma(\vec{z})_1 &= \frac{2981.0}{3130.4} = 0.9523 \\ \sigma(\vec{z})_2 &= \frac{148.4}{3130.4} = 0.0474 \\ \sigma(\vec{z})_3 &= \frac{1.0}{3130.4} = 0.0003\end{aligned}$$



Classification with D.N.N – Modeling



```
model.compile(optimizer='adam',  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

[Cross Entropy]

- Given two random variables X, Y and their respective distributions p, q

$$\begin{aligned}\text{CE}(p, q) &= -\mathbb{E}_{x \sim p}[\log q(x)] \\ &= -\sum_x p(x) \log q(x)\end{aligned}$$

- Why use the loss function to minimize Cross Entropy in classification?
 - ✓ In supervised learning classification problems, we need to ensure that the $\hat{y}$ (q) predicted by machine learning follows a given distribution of labelings y (p).
 - ✓ Take a look into Kullback-Leibler divergence) that is measuring the difference of two distributions

$$\begin{aligned}D_{\text{KL}}(p \parallel q) &= \sum_x p(x) \log \frac{p(x)}{q(x)} \\ &= \sum_x p(x) \log p(x) - \sum_x p(x) \log q(x) \\ &= -H(p) + \text{CE}(p, q)\end{aligned}$$

- ✓ $-H(p)$, that is entropy of p , is fixed,
so CE minimization is has the same purpose of minimization of KL divergence

[Sparse Categorical Crossentropy]

```
import tensorflow as tf
import numpy as np

y=[[1],[2]]
y_pred = [ [0.05, 0.95, 0], [0.1, 0.8, 0.1]]
loss = tf.keras.losses.sparse_categorical_crossentropy(y,y_pred)
print(loss)
```

```
tf.Tensor([0.05129344 2.3025851 ], shape=(2,), dtype=float32)
```

$0 \cdot \ln(0.05) + 1 \cdot \ln(0.95) + 0 \cdot \ln(0)$

$0 \cdot \ln(0.1) + 0 \cdot \ln(0.8) + 1 \cdot \ln(0.1)$

$$\text{Loss} = - \sum_{i=1}^{\text{output size}} y_i \cdot \log \hat{y}_i$$

Multiclass Cross-Entropy



```
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Classification with D.N.N – Modeling

- Training the designed NN model



```
model.fit(train_images, train_labels, epochs=5)
```

Epoch 1/5

1875/1875 [=====] - 3s 2ms/step - loss: 3.6095 -
accuracy: 0.7010

Epoch 2/5

1875/1875 [=====] - 3s 2ms/step - loss: 0.6739 -
accuracy: 0.7566

Epoch 3/5

1875/1875 [=====] - 3s 2ms/step - loss: 0.6113 -
accuracy: 0.7844

Epoch 4/5

1875/1875 [=====] - 3s 2ms/step - loss: 0.5659 -
accuracy: 0.8045

Epoch 5/5

1875/1875 [=====] - 3s 2ms/step - loss: 0.5352 -
accuracy: 0.8150

Classification with D.N.N – Testing

- Testing with test data that is not used in model training



```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
print('\n테스트 정확도:', test_acc)
```

```
313/313 - 0s - loss: 0.6141 - accuracy: 0.7907
```

```
테스트 정확도: 0.7907000184059143
```

Classification with D.N.N – Optimizing the model

- How to improve deep learning model performance
 - ✓ Increase the hidden layer for learning
 - ✓ Increase the Epoch
 - ✓ Use more training data
 - ✓ Tuning hyperparameters
- ✓ Be careful of overfitting!

verbose: Integer. 0, 1, or 2.
Verbosity mode.

0 = silent,
1 = progress bar,
2 = one line per epoch.



```
model = keras.Sequential([
    keras.layers.Flatten(input_shape=(28, 28)),
    keras.layers.Dense(256, activation='relu'),
    keras.layers.Dense(128, activation='relu'),
    keras.layers.Dense(100, activation='relu'),
    keras.layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',\
              loss='sparse_categorical_crossentropy',\
              metrics=['accuracy'])

train_images, test_images = train_images / 255, test_images / 255
model.fit(train_images, train_labels, epochs=10, verbose=1)

test_loss, test_acc = model.evaluate(test_images, test_labels,\
                                     verbose=0)

print('\n테스트 정확도:', test_acc)
```

```
Epoch 1/10
1875/1875 [=====] - 5s 2ms/step - loss: 0.6902 -
accuracy: 0.7752
... <중간 생략> ...
Epoch 10/10
```

```
1875/1875 [=====] - 5s 3ms/step - loss: 0.0713 -
accuracy: 0.9775
```

```
테스트 정확도: 0.9700000286102295
```